

Appearance Manifold Contributor Guide

Runtime weathering interpolation 구조, 코드 책임, 현재 한계와 개선 방향

1. 모듈 역할

AppearanceManifold 모듈은 GammaTon이 만든 풍화 상태를 런타임에서 사용할 수 있는 7D 텐서와 trajectory binary로 변환하고, 두 키 프레임 사이의 중간 텍스처를 계산해 Dynamic Material Instance에 적용합니다.

Layer	Responsibility
GammaTon	Editor 중심 풍화 시뮬레이션과 결과 PNG 생성.
AppearanceManifold	키 프레임 텐서화, trajectory 생성, runtime interpolation, material application.
WeatheringManagerComponent	Blueprint orchestration layer. 파일 경로, 키 프레임 관리, timer-driven playback을 묶습니다.

2. 주요 코드 위치

File	Role
AppearanceManifoldBlueprintFunctionLibrary.h/.cpp	Blueprint callable C++ API 대부분이 위치합니다.
BuildNearestTrajectoryAsync.h/.cpp	nearest trajectory cache를 ThreadPool에서 계산하는 async Blueprint node.
AppearanceManifold.cpp	모듈 startup, Python script path 등록, Python wheel dependency 확인.
Scripts/MainManager.py	Appearance Manifold trajectory generation pipeline entry.
Scripts/Modules/Utility.py	Unreal runtime binary export format 관련 유틸리티.

3. 7D Tensor 데이터 모델

각 픽셀은 7개의 float 채널로 표현됩니다. 모든 C++/Python 경계에서 이 순서를 유지해야 합니다.

Channel	Meaning
0, 1, 2	BaseColor R, G, B
3, 4, 5	Specular R, G, B
6	Roughness
Offset	Pixel i begins at $i * 7$.

4. Binary Format

현재 구현 기준: LoadWeatheringBinaryData가 읽는 runtime trajectory binary는 $[T:\text{int32}][\text{float} * T * 7]$ 구조입니다. 오래된 문서의 $[T][N]$ 헤더 설명과 혼동하지 않는 것이 중요합니다.

Data	Path / Format
TensorCache	Content/TensorCache/[ActorName]_[Index].bin. 키 프레임별 7D tensor 저장.
Trajectory	Content/WeatheringResults 하위. $[T:\text{int32}][\text{float}*T*7]$ runtime sample 저장.
GammaTon source	Saved/GammaTon/Manifold. Base/Spec/Rough PNG source.

5. Blueprint Orchestration

Node / Event	Implementation Role
Set Key Frame Textures	Base/Spec/Rough Texture2D를 Combine Texture Sources로 7D tensor화하고 현재 Count index에 저장합니다.
Set Key Frame Count With Files	index 0부터 TensorCache 파일 존재 여부를 검사해 연속된 파일 수를 Count로 설정합니다.
Get New Key Frame	Saved/GammaTon/Manifold의 PNG를 7D tensor로 읽고 다음 index에 저장합니다.
Clear Key Frame Texture	기존 cache 삭제 후 Sample textures를 keyframe 0으로 재등록합니다.
Generate Weathering Trajectory	중간 keyframe을 복원해 Python pipeline에 전달합니다.
Apply Weathered Texture	nearest cache, interpolation, texture reconstruction, MID parameter update를 수행합니다.

6. Runtime Call Chain

- 1 BeginPlay calls Set Key Frame Count With Files.
- 2 Allow Weathering && KeyWeatheredTexturesCount > 1이면 2초 Delay 뒤 trajectory binary를 로드합니다.
- 3 Trajectory Point Count와 Trajectory Samples를 member variable로 보관합니다.
- 4 초기 keyframe 0/1 tensor를 로드해 Apply Weathered Texture를 호출합니다.
- 5 Weathering Speed 간격으로 Weathering Step Up timer를 반복 실행합니다.
- 6 Weathering Step Up은 Get Key Index 결과로 현재 segment와 Alpha를 계산하고 새 pair를 적용합니다.

7. Apply Weathered Texture Details

- 1 BuildNearestTrajectoryCacheAsync(Tex A 7D, Trajectory Samples, T).
- 2 BuildNearestTrajectoryCacheAsync(Tex B 7D, Trajectory Samples, T).
- 3 InterpolateWeatheringCached(Tex A, Tex B, Cached A, Cached B, Trajectory, T, Alpha).
- 4 ReconstructTexturesFromTensor(Result, Sample Size, Existing Base/Spec/Rough).
- 5 Get Owner -> Get Component By Class(MeshComponent) -> Is Valid.
- 6 ApplyWeatheringTexturesToMesh(MaterialIndex=0, ParamNames=Weathering_Base, Weathering_Spec, Weathering_Rough, ExistingMID=OutMID).

성능상 주의: 현재 Blueprint는 Apply 호출마다 A/B nearest cache를 다시 생성합니다. 키 프레임 pair가 바뀌지 않는 구간에서는 cache를 보관해 재사용하는 개선 여지가 큼니다.

8. C++ API Behavior

Function	Behavior
CombineTextureSources	Base/Spec/Rough UTexture2D를 CPU에서 읽어 7D float array로 결합합니다.
BuildNearestTrajectoryCache	각 픽셀마다 모든 trajectory sample을 brute-force 검색합니다. $O(\text{PixelCount} * T * 7)$.
InterpolateWeatheringCached	nearest index cache와 Alpha를 이용해 중간 7D tensor를 계산합니다.
ReconstructTexturesFromTensor	Transient UTexture2D 3개를 생성하거나 재사용하고 UpdateTextureRegions로 갱신합니다.
ApplyWeatheringTexturesToMesh	MID를 생성/재사용하고 texture parameter 값을 설정합니다.

9. 현재 한계

- nearest search가 brute-force라 trajectory 길이와 texture resolution이 커질수록 비용이 빠르게 증가합니다.
- interpolation은 rounded trajectory index 기반이므로 완전한 continuous interpolation은 아닙니다.
- Material Index 0과 parameter names가 Blueprint에 고정되어 있어 범용성이 제한됩니다.
- Owner에서 첫 MeshComponent를 자동 검색하므로 여러 MeshComponent가 있는 Actor에는 명시적 선택 기능이 필요합니다.
- Transient texture upload가 잦으면 고해상도에서 frame hitch가 발생할 수 있습니다.
- Python pipeline은 editor/runtime packaging 경계를 더 명확히 분리할 필요가 있습니다.

10. 개선 방향

Area	Recommendation
Cache reuse	keyframe pair별 nearest index cache를 보관하고 Alpha만 바뀌는 동안 재사용합니다.
Search acceleration	KD-tree, quantized LUT, segment acceleration structure 등을 검토합니다.
Material flexibility	Material Index와 parameter names를 component 변수로 노출합니다.

Area	Recommendation
Texture update path	Compute Shader 또는 Render Target 기반 GPU path를 검토합니다.
Binary versioning	resolution, channel count, format version header를 추가합니다.
Validation	파일 경로, tensor length, texture size mismatch에 대한 명확한 에러를 반환합니다.